



REPÚBLICA BOLIVARIANA DE VENEZUELA
UNIVERSIDAD NACIONAL EXPERIMENTAL PARA LAS
TELECOMUNICACIONES E INFORMÁTICA (UNETI)
VICERRECTORADO ACADÉMICO
PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA
PROGRAMACIÓN III

EJERCICIO 2

Ionic React con TypeScript

Estudiante: Eliezer Gonzalez
C.I: 30667160

Docente:
Carlos Márquez

Fecha: junio 2026

1. Introducción

El presente informe documenta el desarrollo del Ejercicio 2, correspondiente a la implementación de mejoras interactivas sobre la aplicación, construida con *Ionic React* y *TypeScript*. El ejercicio parte del resultado previo (Ejercicio 1) y extiende la aplicación con nuevas características de interactividad, animaciones, validación de formularios y retroalimentación al usuario.

El enfoque principal fue programar comportamientos personalizados que van más allá de lo que generan automáticamente los *frameworks*, haciendo énfasis en el código escrito manualmente para lograr efectos como el panel de control de partículas, las animaciones al *scroll*, la gamificación y la validación en tiempo real del formulario.

2. Tecnologías Utilizadas

Además del *stack* base del Ejercicio 1 (*React*, *TypeScript*, *Ionic Framework*, *Framer Motion*), se incorporaron las siguientes tecnologías y *APIs* adicionales:

2.1 Context API de React

Se implementó un contexto global (*ThemeContext*) para manejar el estado del tema oscuro/claro a lo largo de toda la aplicación, permitiendo que cualquier componente acceda y modifique el tema sin necesidad de *prop drilling*.

2.2 Intersection Observer API

API nativo del navegador utilizada a través del *hook* personalizado *useScrollAnimation* para detectar cuándo los elementos del DOM entran al *viewport* y disparar animaciones de aparición al hacer *scroll*.

2.3 @capacitor/haptics

Librería de Capacitor para acceder a la vibración del dispositivo. Se implementó a través del *hook* *useHaptic*, que proporciona *feedback* táctil al usuario en diversas interacciones. En dispositivos sin soporte, la llamada simplemente no tiene efecto.

2.4 Custom Hooks

Se crearon cuatro *hooks* personalizados reutilizables que encapsulan lógica específica:

- *useScrollAnimation*: animaciones basadas en visibilidad con *Intersection Observer*
- *useHaptic*: abstracción del *feedback* de vibración con Capacitor
- *useGamification*: rastreo de métricas de interacción del usuario
- Lógica de partículas extendida en *MouseTrail*

3. Arquitectura de Archivos

A continuación, se detallan los archivos creados y modificados durante el ejercicio, diferenciando entre nueva infraestructura y modificaciones sobre componentes existentes.

3.1 Archivos Nuevos Creados

- `src/context/ThemeContext.tsx`

- **Propósito:** Contexto global de *React* para gestionar el tema oscuro/claro
- **Lo que se programó:** *createContext*, *useContext*, *useState*, aplicación de clases CSS al *body* del documento
- src/hooks/useScrollAnimation.ts
 - **Propósito:** Hook para activar animaciones al hacer *scroll*
 - **Lo que se programó:** *IntersectionObserver* con *threshold* configurable, *ref* de elemento, estado booleano de visibilidad
- src/hooks/useHaptic.ts
 - **Propósito:** Abstracción de vibración para dispositivos móviles
 - **Lo que se programó:** Llamadas a *Haptics.impact()* y *Haptics.vibrate()* con manejo de error silencioso
- src/hooks/useGamification.ts
 - **Propósito:** Rastreo de métricas: partículas activadas y distancia del mouse
 - **Lo que se programó:** *useRef* para posición previa, *useState* para contadores, *listener* de *mousemove* con cálculo de distancia euclidiana
- src/components/ParticleControls.tsx / ParticleControls.css
 - **Propósito:** Panel flotante de control del sistema de partículas
 - **Lo que se programó:** Componente con sliders *range* controlados, *toggles* booleanos, *callbacks* hacia el componente padre, posicionamiento fijo en esquina inferior derecha

3.2 Archivos Modificados

- `src/components/MouseTrail.tsx` — Extensión mayor del sistema de partículas
- `src/pages/Home.tsx` — *Toggle* de tema, scroll animations, hover effects
- `src/pages/Profile.tsx` — *Skills* animadas, *timeline*, *social links*, *avatar parallax*
- `src/pages/Contact.tsx` — Validación en tiempo real, contador de caracteres, envío simulado
- `src/components/Menu.tsx` — *Haptic feedback* en navegación
- `src/App.tsx` — *ThemeProvider*, estado global de partículas, gamificación
- `src/theme/antigravity.css` — Variables CSS para modo oscuro y claro

4. Funcionalidades Implementadas

Se detallan a continuación las quince funcionalidades programadas, con énfasis en el código escrito y las decisiones técnicas tomadas.

4.1 Panel de Control de Partículas

Se programó el componente *ParticleControls.tsx* con estado controlado para cada parámetro del sistema de partículas. El panel se posiciona de manera fija en la esquina inferior derecha mediante CSS position: *fixed*. Los valores se comunican al componente padre (*App.tsx*) a través de *callbacks*, siguiendo el patrón de *lifting state up* de *React*.

Parámetros controlables programados:

- Modo de interacción: repulsión / atracción (*toggle* booleano)
- Gravedad: activar / desactivar caída de partículas (*toggle* booleano)

- Cantidad de partículas: slider de 20 a 150 unidades
- Velocidad de movimiento: multiplicador de 0.1x a 2x
- Radio de interacción con el mouse: 50px a 300px
- Botón de *reset* a valores predeterminados

4.2 Sistema de Partículas Extendido

Se extendió el componente `MouseTrail.tsx` con tres nuevas características programadas desde cero:

Modo Atracción / Repulsión

Se agregó lógica condicional en el *loop* de animación para invertir el vector de fuerza sobre cada partícula según el modo activo. En repulsión, el vector apunta desde el *mouse* hacia afuera; en atracción, apunta hacia el *mouse*.

Explosiones al *Click*

Se implementó un *listener* de *click* que genera un conjunto de partículas temporales con velocidades iniciales radiales, calculadas distribuyendo ángulos uniformemente (360° / cantidad de partículas). Cada partícula temporal tiene una vida útil limitada y se elimina al agotarla.

Física de Gravedad

Se agregó una aceleración vertical constante (gravedad) que se aplica opcionalmente al vector de velocidad Y de cada partícula cuando el *toggle* está activo.

4.3 Modo Oscuro / Claro

Se programó *ThemeContext* con *React.createContext* y un *Provider* que envuelve toda la aplicación en `App.tsx`. El *toggle* aplica la clase CSS *dark* o *light* al elemento *body*, lo que activa las variables CSS correspondientes definidas en `antigravity.css`. El color de las partículas también cambia según el tema activo.

4.4 Animaciones al *Scroll*

El *hook useScrollAnimation* recibe una referencia de elemento y un *threshold* (por defecto 0.1). Internamente crea un *IntersectionObserver* que actualiza un estado booleano *isVisible* cuando el elemento entra al *viewport*. Este estado se usa para aplicar clases CSS con transiciones de opacidad y *translateY*.

Se aplicó en: sección *hero* de *Home*, avatar y tarjetas de *Profile*, formulario de *Contact*.

4.5 Validación de Formulario en Tiempo Real

En *Contact.tsx* se programó un sistema de validación que se ejecuta en el evento *onChange* de cada campo. Las reglas implementadas son:

- Nombre y Asunto: campo obligatorio (longitud > 0)
- Mensaje: mínimo 10 caracteres
- Formato de email preparado con *regex* para uso futuro

Los errores se almacenan en un objeto de estado (*errors*) y se muestran con animaciones de *Framer Motion* debajo de cada campo. El formulario sólo permite el envío si todos los campos son válidos.

4.6 Contador de Caracteres Animado

Se programó un contador en el *textarea* del mensaje que calcula en tiempo real (*value.length*) cuántos caracteres lleva el usuario. El color del contador cambia mediante lógica condicional:

- Naranja/amarillo: menos de 10 caracteres (no cumple el mínimo)
- Verde: 10 o más caracteres (requisito cumplido)

4.7 Envío Simulado con *Feedback*

Se programó un flujo de envío con estados visuales diferenciados:

1. El usuario hace *click* en Enviar — el botón cambia a estado *loading* (*isLoading*: true)
2. *setTimeout* de 2 segundos simula latencia de red
3. *isSuccess*: true muestra mensaje de confirmación animado
4. Después de 3 segundos, se limpia el formulario y resetean todos los estados

4.8 Haptic Feedback

El *hook useHaptic* encapsula las llamadas a *@capacitor/haptics* con un *try/catch* silencioso. Se registra vibración en:

- *Click en toggle* de tema oscuro/claro
- Navegación entre páginas en el menú lateral
- *Click* en botón de envío del formulario
- *Click* en *social links* de *GitHub* y *LinkedIn*

4.9 Gamificación

El *hook useGamification* implementa dos contadores sin persistencia (sin *localStorage*):

- Partículas activadas: se incrementa cada vez que el usuario genera una explosión con *click*
- Distancia del *mouse*: se calcula con $Math.sqrt((dx)^2 + (dy)^2)$ acumulando la distancia entre posiciones consecutivas del *mouse*

Estos datos se muestran en el panel de control de partículas en tiempo real.

4.10 Perfil Mejorado

Skills Animadas

Se programó una sección de habilidades con barras de progreso que se activan cuando la sección entra al *viewport* (usando *useScrollAnimation*).

Cada barra tiene un *delay* diferente para crear efecto secuencial. Las 8 tecnologías y sus niveles son:

- *React* 90% — *TypeScript* 85% — *Ionic* 80% — *Flutter* 75%
- *Django* 70% — *Node.js* 85% — *PostgreSQL* 65% — *Git* 80%

Timeline Vertical

Se programó una línea de tiempo con puntos decorativos y animaciones de entrada secuenciales usando *Framer Motion*. Los datos son *placeholder* editables: 2024 (Desarrollador *Full Stack*), 2023 (Especialización *Frontend*), 2022 (Inicios en Desarrollo).

Social Links

Se agregaron botones para GitHub y LinkedIn con efectos *hover* y *tap* programados con *Framer Motion* (*whileHover*, *whileTap*). Las URLs son *placeholder* y el *haptic feedback* se activa al hacer *click*.

Avatar Interactivo

El avatar de perfil tiene efecto de rotación suave al *hover* y se agranda ligeramente, programado con *variants* de *Framer Motion* y la propiedad *rotate*.

5. Procedimiento de Implementación

La implementación siguió un orden por fases para evitar dependencias circulares y poder probar cada característica de forma incremental:

5. Fase 1 — Infraestructura: *ThemeContext*, *hooks* (*useScrollAnimation*, *useHaptic*, *useGamification*)
6. Fase 2 — *MouseTrail*: modos de interacción, explosiones, gravedad, colores dinámicos
7. Fase 3 — *ParticleControls*: panel flotante con *sliders*, *toggles* y estadísticas

8. Fase 4 — Páginas: *scroll animations, hover effects, toggle* de tema en todas las páginas
9. Fase 5 — Formulario *Contact*: validación en tiempo real, contador, envío simulado
10. Fase 6 — Perfil *Profile*: *skills, timeline, social links, avatar* interactivo
11. Fase 7 — *App.tsx*: integración de *ThemeProvider*, estados globales de partículas, gamificación

6. Instrucciones de Ejecución

Para levantar la aplicación y probar todas las funcionalidades:

npm install (*instalar dependencias*)

npm run dev (*levantar servidor de desarrollo*)

Repositorio del proyecto: <https://github.com/Elagonzalez/E1PG3.git>

Funcionalidades a verificar en el navegador:

- Panel de partículas: *click* en ícono de engranaje (esquina inferior derecha)
- Modos de interacción: cambiar entre repulsión y atracción en el panel
- Explosiones: *click* en cualquier parte de la pantalla
- Gravedad: *toggle* en el panel de control
- Tema oscuro/claro: ícono luna/sol en el *header* de cualquier página
- *Scroll* animations: hacer scroll en cada página
- *Hover* en tarjetas: pasar el mouse sobre las *glass-cards*
- Formulario: intentar enviar vacío o con menos de 10 caracteres en el mensaje

- *Social links*: hover sobre GitHub y LinkedIn en el perfil
- *Haptic*: interacciones en dispositivo móvil

7. Aprendizajes Obtenidos

Este ejercicio permitió profundizar en los siguientes conceptos técnicos:

- **Context API**

Comprender la diferencia entre estado local y estado global, y cuándo es apropiado usar un contexto. Implementar *ThemeContext* enseñó el patrón *Provider/Consumer* y cómo evitar *prop drilling* en aplicaciones medianas.

- **Custom Hooks**

La creación de *hooks* reutilizables (*useScrollAnimation*, *useHaptic*, *useGamification*) reforzó el principio de separación de responsabilidades. Cada *hook* encapsula una lógica específica y puede reutilizarse en cualquier componente.

- **Intersection Observer**

Implementar esta *API* directamente (sin librerías externas) permitió entender su funcionamiento interno: el *callback* recibe un *array* de entradas, cada una con la propiedad *isIntersecting* que indica si el elemento está en el *viewport*.

- **Framer Motion avanzado**

Se aprendió a combinar *variants*, *whileHover*, *whileTap*, *AnimatePresence* y *transition* de forma coordinada para crear animaciones complejas que se sienten naturales. El balance entre *duration* y *delay* en secuencias es clave.

- **Gestión de estado en física de partículas**

Extender el sistema de partículas con nuevos modos, gravedad y explosiones requirió manejar múltiples *arrays* de partículas temporales y permanentes simultáneamente, usando *useRef* para evitar *re-renders* innecesarios en el *loop* de animación.

- **Validación de formularios sin librerías**

Implementar validación en tiempo real desde cero (sin *Formik* ni *React Hook Form*) reforzó la comprensión de eventos *onChange*, estado de errores y cuándo mostrar *feedback* al usuario.

8. Conclusiones

El Ejercicio 2 permitió transformar una aplicación funcional en una experiencia interactiva y dinámica, implementando quince características nuevas que van desde el control de un sistema de partículas hasta validación de formularios y gamificación básica.

El aspecto más desafiante fue la integración del estado global de gamificación con el panel de control, que requirió reestructurar *App.tsx* para tener un componente interno que accediera al *hook useGamification* y pasara los datos hacia *ParticleControls*.

El resultado es una aplicación que demuestra el uso de múltiples conceptos avanzados de *React* (contextos, *hooks* personalizados, *Intersection Observer*, *Framer Motion*) de forma integrada y coherente, cumpliendo con el criterio de enfocarse en lo programado más allá de lo que generan automáticamente los *frameworks*.